

DATE : 25.09.2026
DT/NT : DT
LESSON : LINUX
SUBJECT: LINUX ENVIRONMENT
VARIABLES
SESSION : 5
BATCH : B 424-020DE

AWS-DEVOPS



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59



Table of Contents

- ▶ What are Environment Variables?
- ▶ Common Environment Variables
- ▶ Accessing the Variables
- ▶ The PATH Variable
- ▶ Quoting with Variables

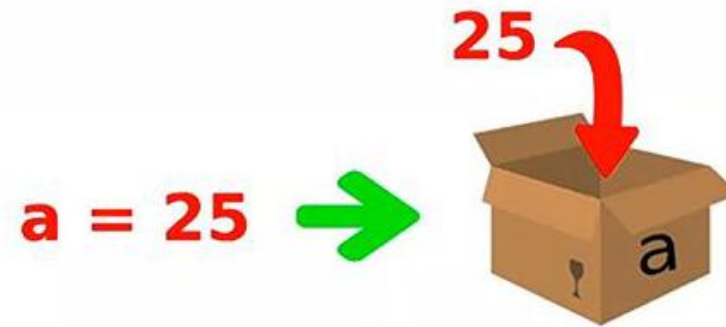




What are Environment Variables?



Variables



Variables are used to store for data values

What are Environment Variables?



- **"Environment variable"** is a value that can be stored, used, and modified by a computer operating system or an application. These variables can include configuration information, paths, language settings, and other data that influence the behavior of the system or application.

What are Environment Variables?



- A list of all defined environment variables can be viewed by entering the `env` command. There is nothing special about the names of the variables, but traditionally, environment variables should have **UPPERCASE** names.

Common Environment Variables



Common Environment Variables



Variable	Description
PATH	This variable contains a list (separated by colons :) of directories that your system searches for executable files.
USER	The username of the current user.
HOME	The default path to the user's home directory.
EDITOR	The path to the program used for editing file contents.
UID	The unique identifier of the user.
TERM	The default terminal emulator setting.
SHELL	The shell used by the user (e.g., <code>bash</code> , <code>zsh</code>).
LANG	The current locale/language settings.

Common Commands



Command	Description
env	A shell command used to view or modify environment variables. It's commonly used to list all environment variables.
printenv	Prints all environment variables or a specific one if provided.
set	Sets or unsets shell variables . When used without arguments, it displays a list of all shell and environment variables along with shell functions.
unset	Deletes/removes shell or environment variables .
export	Defines or makes a variable available to child processes (i.e., sets environment variables).



Accessing Variable



Common Commands



Command	Description
<code>echo \$VARIABLE</code>	To display value of a variable
<code>env</code>	Displays all environment variables
<code>VARIABLE_NAME=variable_value</code>	Create a new shell variable
<code>unset</code>	Remove a variable
<code>export Variable=value</code>	To set value of an environment variable

Accessing Variable



printenv or echo Display Path Environment Variable.

```
linuxuser@techproeducation: ~$ printenv USER
linuxuser
linuxuser@techproeducation: ~$ printenv HOME
/home/linuxuser
linuxuser@techproeducation: ~$ echo $USER
linuxuser
linuxuser@techproeducation: ~$ echo $HOME
/home/linuxuser
linuxuser@techproeducation: ~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
linuxuser@techproeducation: ~$ |
```

Define a New Variable



```
linuxuser@techproeducation: ~$ newvariable=newvalue
linuxuser@techproeducation: ~$ echo $newvariable
newvalue
linuxuser@techproeducation: ~$ |
```

Remove a Variable



unset Remove a variable from the system.

```
linuxuser@techproeducation: ~$ newvariable=newvalue
linuxuser@techproeducation: ~$ echo $newvariable
newvalue
linuxuser@techproeducation: ~$ unset newvariable
linuxuser@techproeducation: ~$ echo $newvariable

linuxuser@techproeducation: ~$ |
```

Modern Command Substitution

Old Style

```
DATE=`date`
```

Recommended

```
DATE=$(date)
```

```
echo "$DATE"
```

Shell Variables vs Environment Variables

Shell Variable

- Current shell only
- Not inherited by child processes

```
MYVAR="Linux"  
echo $MYVAR
```

Environment Variable

- Exported variable
- Available to child processes

```
export MYVAR="Linux"  
bash  
echo $MYVAR
```

Environment Variables in DevOps

AWS Configuration

AWS_REGION
AWS_PROFILE
AWS_ACCESS_KEY_ID

Application Configuration

DATABASE_URL
DB_PASSWORD
API_KEY

Why?

Configuration outside code
Easier deployment
Better security

Temporary vs Permanent Variables

Temporary

```
export COURSE="AWS-DevOps"
```

Only current terminal session

Permanent

Add to ~/.bashrc

```
source ~/.bashrc
```

Environment Variables in Containers

Docker

```
docker run -e DATABASE_URL=value myapp
```

Kubernetes

env:

```
- name: DB_HOST  
  value: mysql-service
```

Python Environment Variables

Example

```
import os

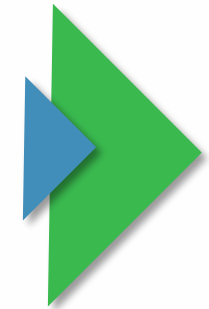
user=os.getenv("USER")
home=os.getenv("HOME")

print(user)
print(home)
```

Exercise



1. Create a variable named **MYVAR** with the value "my value".
2. Print the value of the **MYVAR** variable to the screen.
3. Assign "new value" to the **MYVAR** variable.
4. Print the value of the **MYVAR** variable to the screen.
5. Delete the **MYVAR** variable.
6. Print the value of the **MYVAR** variable to the screen.



The PATH Variable



The PATH Variable



- When we want the system to run a command, we almost never need to provide the full path to that command.
- For example, we know that the `ls` command is located in the `/bin` directory (you can verify this with the `which ls` command), but we don't have to type `/bin/ls` for the computer to list the contents of the current directory.
- This convenience is provided by the **PATH environment variable**. This variable lists all the directories on the system where executable files may be found.

The PATH Variable



printenv Display Path Environment Variable.

```
linuxuser@techproeducation: ~$ printenv PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
linuxuser@techproeducation: ~$
```

In this example, the directories `/usr/local/sbin`, `/usr/local/bin`, `/usr/sbin`, `/usr/bin`, `/sbin`, and `/bin` are searched one by one for the required program. Even if not all directories in the PATH variable are searched, the search will **stop as soon as a match is found**.

The PATH Variable



export Add a New Directory to the Path.

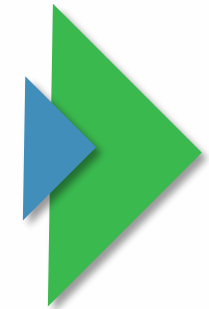
```
linuxuser@techproeducation: ~$ export PATH=$PATH:games/awesome
linuxuser@techproeducation: ~$ printenv PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:games/awesome
linuxuser@techproeducation: ~$ |
```

Let's say you want to run a file called fun. When you use the find command, you discover that it's located in a directory named /games/awesome. However, /games/awesome is **not in your PATH**, and you don't want to type the full path every time just to run fun. So, you can **add it to your PATH variable using the export command**.

PATH Variable: Create Your Own Command

Workflow

1. Create ~/bin
2. Create script
3. chmod +x script
4. Add PATH
5. Run command anywhere



Quoting with Variables



Quoting



- **Quoting** is used to disable the special processing of certain characters and words, to **prevent parameter expansion**, and to **preserve quoted values**.
The **Bash shell** recognizes certain rare but important characters as **special**.

```
echo "$PATH"  
echo "$PS1"
```

Quoting



- Double quotes ("quote") preserve everything between the two double quotation marks **except** the characters: \$, ', " and \.
- Single quotes ('quote') preserve **everything** between the two single quotation marks **exactly as written**.
- Use the **backslash** (\) to change the special meaning of characters or to **escape** special characters like quotation marks within text.

```
echo "$SHELL"  
echo "/etc/*.conf"
```

```
echo '$SHELL'  
echo '/etc/*.conf'
```

```
echo "Path is  
\$PATH"
```

Quoting Variables in Bash

Double Quotes

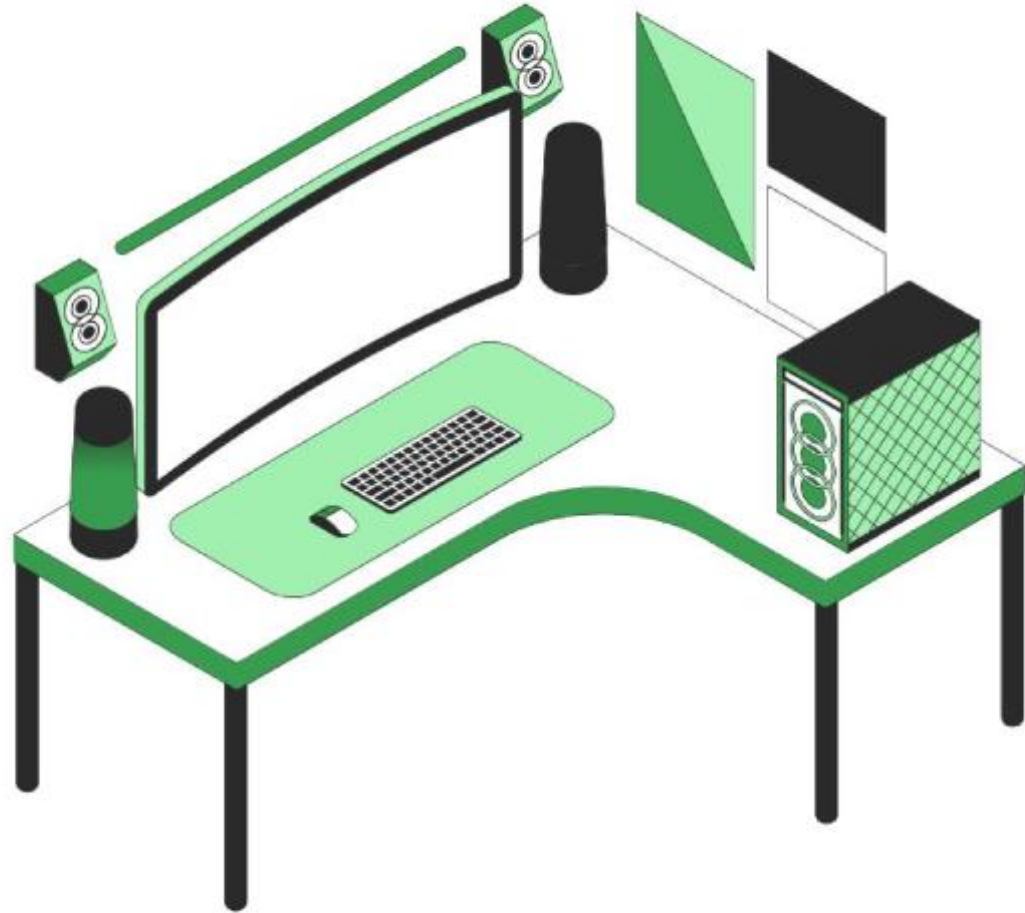
```
echo "Hello $USER"
```

Variables expand

Single Quotes

```
echo 'Hello $USER'
```

Text stays unchanged



Do you have any questions?

Send it to us! We hope you learned something new.